

Optimizing Autonomous Fleet Logistics via Proximal Policy Optimization

Aksel Kretsinger-Walters, Alena Chan, Andrey Aksyutkin

December 2025

Abstract

This report details the application of Reinforcement Learning (RL) to the complex logistics of autonomous robotaxi fleets. We model the city environment as a dynamic graph and utilize Proximal Policy Optimization (PPO) to maximize fleet profitability and demand satisfaction. By encoding the city as a network of connected nodes—representing vehicle availability and demand hotspots—we demonstrate that PPO with clipped updates generates stable and profitable policies compared to baseline heuristics. We further analyze the impact of high-reward "red nodes" and their adjacent "orange nodes" on agent behavior within our simulation.

1 Introduction

One of the most promising and revolutionary applications of reinforcement learning (RL) is in the domain of autonomous logistics, specifically self-driving car fleets. The challenges facing the autonomous vehicle community span multiple dimensions, including intellectual, ethical, financial, and technical hurdles. In this project, we narrow our focus to optimizing the operational profitability of robo-taxi fleets.

The relevance of this domain is underscored by its enormous commercial and consumer potential. Commercially, applications extend to freight, last-mile delivery, and industrial logistics. On the consumer side, the technology enables robotaxis, improved public transportation, and critical response systems that enhance safety and accessibility. Effectively managing such fleets solves key problems in inventory management and urban planning.

However, monitoring, maintaining, and optimizing a large fleet of self-driving cars is a non-trivial problem. Operators must maximize availability, minimize downtime, and ensure network resilience. Furthermore, they must accurately predict supply and demand to facilitate dynamic pricing and maximize utilization efficiency. While tasks such as scheduling maintenance, recharging vehicles, and minimizing wait times are difficult individually, jointly optimizing across all these dimensions while adapting to distribution shifts is significantly more challenging. This interactive, multi-objective nature makes the problem a natural candidate for reinforcement learning and agentic approaches.

2 Methodology

2.1 RL Formulation

We simulate a fleet of self-driving cars as a Markov Decision Process (MDP) to develop algorithms that optimize operations and profitability.

- **State Space:** The state includes the location of each car, the distribution of passenger demand, and current weather conditions. We include weather as it has been shown that meteorological factors significantly drive the location and volume of demand, accounting for substantial variance in ride-hailing profits.
- **Action Space:** The action space consists of directing each car to move in one of the cardinal directions (North, South, East, West) or stay put.

While the problem could be approached by having each car make location decisions independently, implicit coordination often makes such solutions unstable. Therefore, we utilize a centralized policy approach.

2.2 Environment and Simulation

We modeled the city as a graph of nodes representing pickup and drop-off locations, with edges representing the roads connecting them. Leveraging the regular structure of cities like New York, we utilized a grid representation where road distances are estimated using Manhattan distance. To limit training time—since the state-action space scales linearly with grid size—we initially evaluated basic policies on a simplified 4×5 graph before scaling to larger simulations.

Gymnasium Framework We built our simulation environment using the OpenAI Gymnasium scaffolding. Leveraging Gymnasium abstracted many implementation details, allowing us to focus on defining the state space, action space, transition model, and reward structures as follows.

State Space The state space was deliberately abstracted away from the graph it repre

2.3 Proximal Policy Optimization (PPO)

For the fleet distribution algorithm, we utilize Proximal Policy Optimization (PPO) (Schulman et al. 2017). PPO is a variation of the actor-critic architecture that updates the policy using a clipped surrogate objective, denoted as $L_t(\theta)$. This clipping is vital for real-time business decisions, as it prevents large, destabilizing updates that could, for example, erroneously direct the entire fleet to a low-demand area.

Stable updates are achieved by maintaining "old" and "current" policy distributions over the state-action space and limiting the divergence between them. The objective function we maximize via gradient ascent is:

$$L_t(\theta) = \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right)$$

where ϵ bounds the divergence (typically $\epsilon \in [0.1, 0.3]$), $r_t(\theta)$ is the probability ratio of the action under the current vs. old policy, and $\hat{A}_t$ is the estimated advantage. We found $\epsilon = 0.2$ provided the most stable and accurate policy.

Our policy model is a 2-layer neural network with a hidden size of 128 and ReLU activation, optimized using Adam. The critic (Value Function) shares this architecture but outputs a single value (advantage), optimized via mean squared error.

2.4 Reward Function

Our objective is to maximize the number of matched requests (served demand). The reward function incentivizes three behaviors:

1. Minimizing unnecessary movement (fuel/time costs).
2. Relocating free cars to forecasted high-demand areas (pre-positioning).
3. Successfully picking up nearby customers.

3 Results

We compared different policies on a 20-node city grid. To test robustness, we simulated a high-demand center (controlled by parameter λ) moving in a circular trajectory around the city, the position of which was predictable via the weather state provided to the agent. This assumption is realistic, as autonomous fleets (e.g., Waymo) have access to real-time weather and event forecasting.

Evaluations were conducted over 20-minute episodes, reflecting the average ride time and the threshold at which customers typically cancel if a vehicle has not arrived.

3.1 Policy Gradient vs. Baseline

As a baseline, we used a **NEAREST** car assignment heuristic, where the closest available car is dispatched to a request. If no demand exists, spare cars move randomly.

We observed that the Policy Gradient approach, which proactively relocates cars and intelligently passes on suboptimal immediate demand for greater long-term rewards, significantly outperformed the baseline.

- **NEAREST Baseline Profit:** 7,990.71 units
- **Policy Gradient Profit:** 12,829.93 units
- **Improvement:** $\approx 60\%$

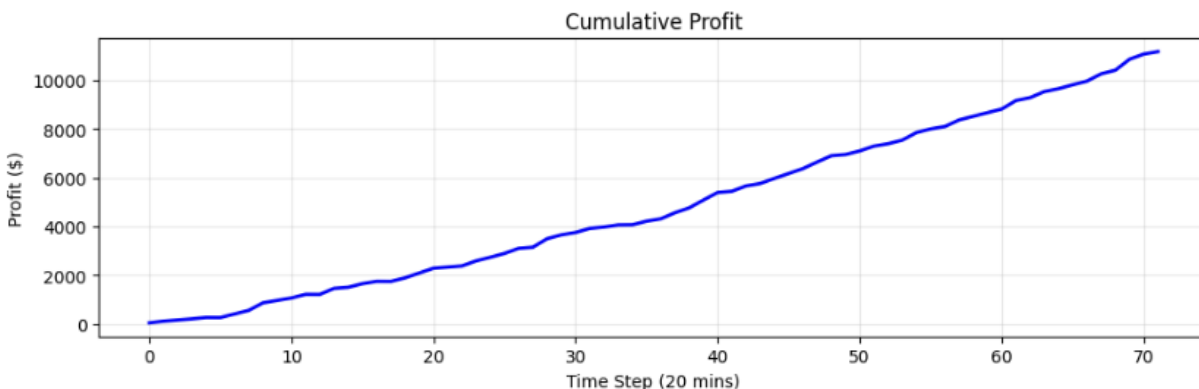


Figure 1: Cumulative profit over a 20-minute episode comparing Policy Gradient to Baseline.

3.2 Policy Gradient vs. PPO (Clipped)

We further compared vanilla Vanilla Policy Gradient (VPG) against PPO with clipped updates on the 20-node grid with circular demand movement.

While PPO with clipping requires longer training times (more iterations) to learn because it bounds aggressive policy updates, it ultimately achieves higher cumulative rewards and improved stability. This stability is crucial for complex environments where VPG might collapse into suboptimal deterministic policies. Our observations align with theoretical findings in Wang, Zhang, and Gao 2025, which suggest similar long-term performance between clipped PPO and repeated VPG, though PPO offers safety guarantees essential for physical deployment.

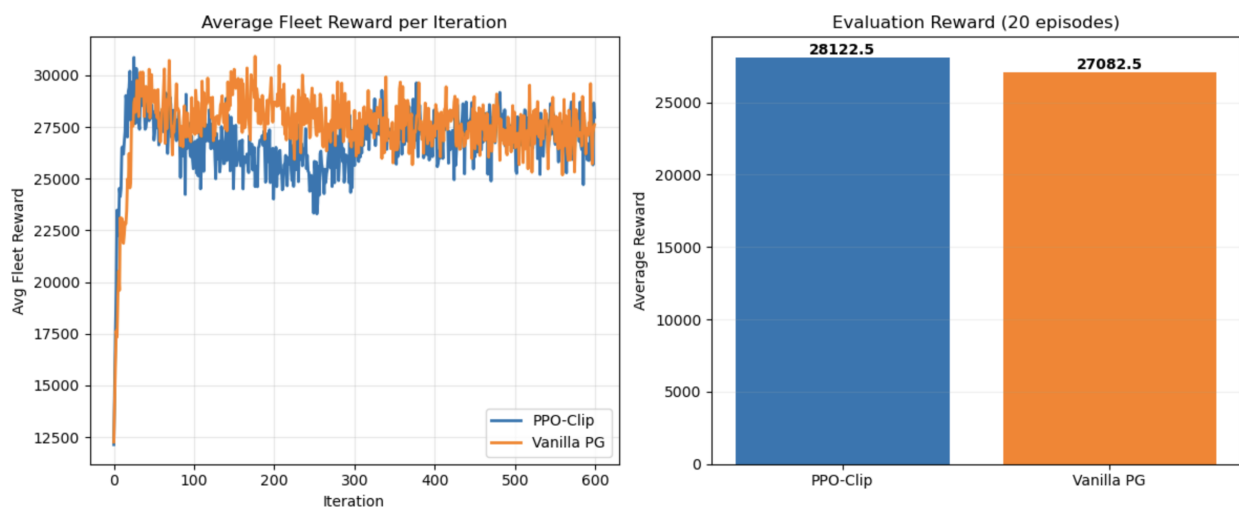


Figure 2: Training curves: Gradient policy update vs. PPO with clipping.

4 Conclusion

In this project, we successfully formulated the autonomous fleet management problem as an MDP and solved it using PPO. By explicitly modeling the city as a graph of nodes containing vehicle count data and highlighting high-reward zones (Red/Orange nodes), our agent learned to anticipate demand rather than merely reacting to it.

The code is available at <https://github.com/akseldkw07/Waymo-Agent.git>.

A demo video is available at <https://youtu.be/IXpzuxEX3mU>.

5 Future Work

Our solution could be extended to other cities with irregular layouts. This would require reinterpreting the graph structure, as the Manhattan distance heuristic applies primarily to grid-like cities. Future work would involve integrating OpenStreetMap data (OSMnx) to calculate realistic travel times on non-grid road networks.

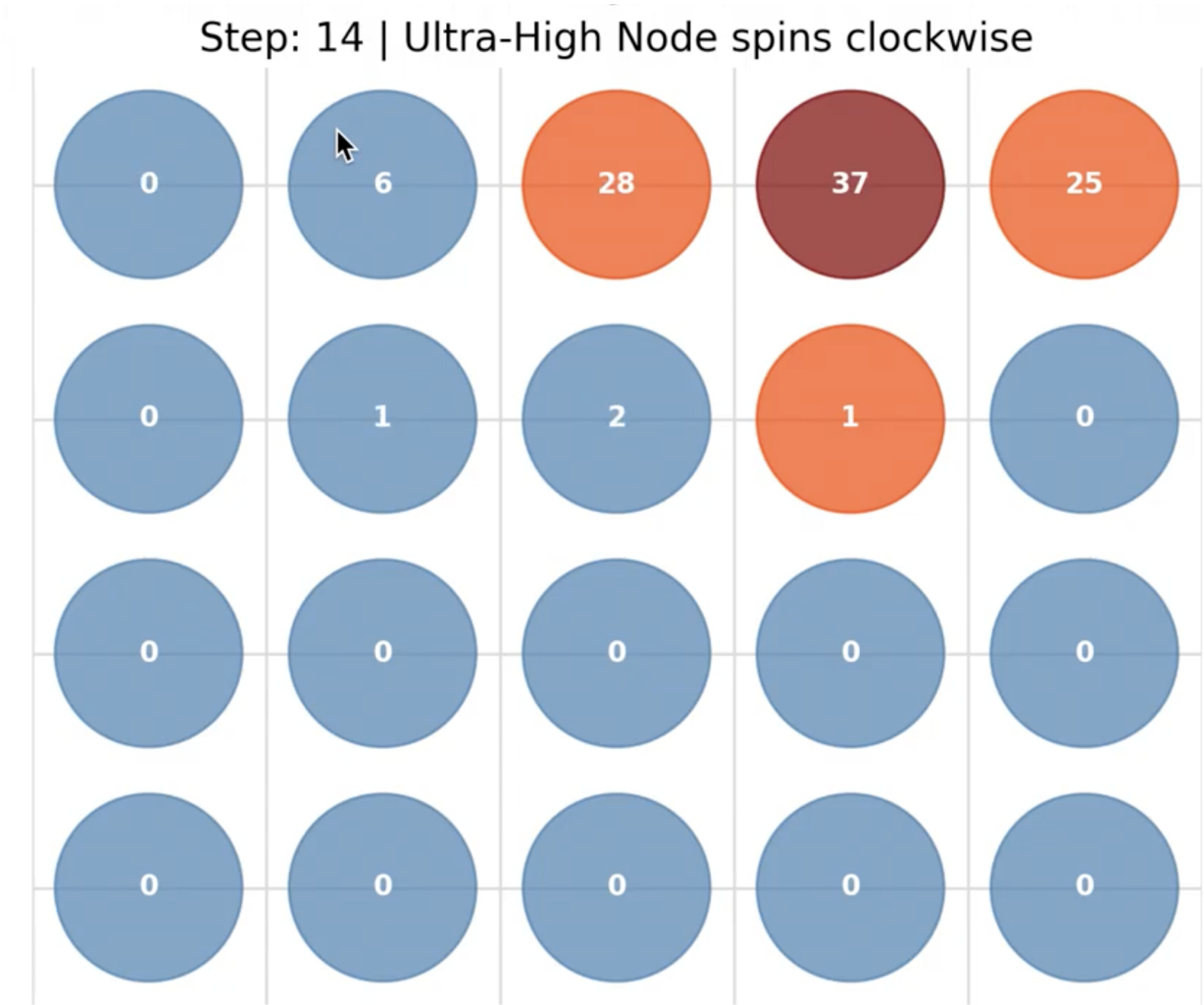


Figure 3: Visualization of trained behavior: Cars aggregating near high-value Red/Orange nodes.

References

- Schulman, John et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- Wang, Tao, Ruipeng Zhang, and Sicun Gao (2025). “Improving Value Estimation Critically Enhances Vanilla Policy Gradient”. In: *arXiv preprint arXiv:2505.19247*. arXiv: 2505.19247. URL: <https://arxiv.org/abs/2505.19247>.